Міністерство освіти і науки України

Львівський національний університет імені Івана Франка

Факультет електроніки та комп’ютерних технологій

Кафедра радіоелектроніки та комп’ютерних систем

Звіт

Про виконання лабораторної роботи №4

З курсу “**Машинне навчання**”

«Дослідження дерев рішень та ансамблевих методів у задачі багатокласової класифікації за наявності пропущених значень»

Виконав:

студент групи ФЕІ-44с

Мариняк А. П.

Перевірив:

доцент Сінькевич О.О.

Львів 2026

Мета роботи: Дослідити та порівняти три алгоритми класифікації - дерево рішень (Decision Tree), випадковий ліс (Random Forest) та градієнтний бустинг (XGBoost) - на реальному датасеті Annealing, що містить пропущені значення.

Основну увагу приділити впливу способу обробки пропусків на якість класифікації, коректному (без витоку даних) підбору гіперпараметрів за допомогою Optuna, неупередженому оцінюванню моделей (вкладена крос-валідація) та статистичному порівнянню результатів, а також інтерпретації моделей і розбіжностей між їхніми прогнозами (permutation importance, SHAP)

Завдання:

1. Завантаження та первинний аналіз даних: завантажити датасет Annealing, визначити кількість спостережень та типи ознак, обчислити частку пропусків і видалити константні/порожні ознаки, а також проаналізувати розподіл цільової змінної та механізм виникнення пропусків (MCAR/MAR/MNAR).

2. Поділ даних і пайплайн обробки: виконати стратифікований поділ даних і побудувати два пайплайни обробки в scikit-learn (базова імпутація та імпутація з бінарними індикаторами пропусків), що виключають витік даних.

3. Навчання базових моделей: навчити моделі Decision Tree, Random Forest та XGBoost з типовими гіперпараметрами для обох варіантів обробки та оцінити їх за допомогою повторної стратифікованої крос-валідації.

4. Підбір гіперпараметрів та оцінка: виконати оптимізацію гіперпараметрів для найкращого варіанту за допомогою Optuna, використовуючи вкладену крос-валідацію для отримання неупередженої оцінки якості моделей, та порівняти її з оптимістичним best_value.

5. Статистичне порівняння: застосувати скоригований t-тест Наде-Бенжіо для перевірки статистичної значущості різниці в якості моделей між двома методами обробки пропусків та між найкращими моделями.

6. Фінальна оцінка та інтерпретація: оцінити найкращі моделі на тестовій вибірці (матриці плутанини, classification report), обчислити важливість ознак (MDI та Permutation Importance) і проаналізувати об'єкти розбіжностей між моделями за допомогою SHAP-значень

ТЕОРЕТИЧНІ ВІДОМОСТІ

Механізми виникнення пропусків

За класифікацією Д. Рубіна розрізняють три основні механізми виникнення пропущених значень у даних:

- MCAR (Missing Completely At Random): ймовірність пропуску абсолютно випадкова і не залежить ані від спостережуваних, ані від неспостережуваних (відсутніх) значень ознак.
- MAR (Missing At Random): ймовірність пропуску залежить лише від інших, наявних (спостережуваних) ознак у датасеті, але не від самого відсутнього значення.
- MNAR (Missing Not At Random): ймовірність пропуску безпосередньо залежить від самого відсутнього значення або від інших неспостережуваних чинників, які не зафіксовані в даних. Якщо механізм не є MCAR (тобто це MAR або MNAR), сам факт відсутності значення може нести важливу інформацію для передбачення цільової змінної. У таких випадках додавання бінарних індикаторів пропусків (окремих колонок, що фіксують факт наявності чи відсутності даних) здатне суттєво поліпшити якість класифікації.

Оцінка моделей та статистичне порівняння

Під час підбору гіперпараметрів метрика, отримана як максимум на крос-валідації (наприклад, best_value в Optuna), є оптимістично зміщеною, оскільки модель фактично адаптується до валідаційних даних.

Для отримання чесної (неупередженої) оцінки якості використовується вкладена крос-валідація (nested CV), де підбір параметрів відбувається лише на тренувальній частині внутрішніх блоків, а фінальна оцінка - на незалежній валідаційній частині зовнішніх блоків

Звичайний t-тест Стьюдента непридатний для порівняння результатів крос-валідації, оскільки оцінки на різних блоках перетинаються (мають спільні тренувальні дані) і не є незалежними, що призводить до недооцінки дисперсії.

Для коректного статистичного порівняння використовується скоригований t-тест Наде-Бенжіо, який вносить поправку в знаменник, враховуючи відношення розмірів тестової та тренувальної вибірок.

Інтерпретація моделей

Для розуміння логіки прийняття рішень моделями використовують два основні підходи до оцінки важливості ознак:

- MDI (Mean Decrease Impurity): внутрішня метрика деревовидних моделей, яка оцінює, наскільки кожна ознака зменшила неоднорідність (ентропію або джині) під час побудови дерев. Вона обчислюється на вже оброблених ознаках і може бути схильна до переоцінки ознак із великою кількістю унікальних значень.
- Permutation Importance: моделе-незалежний метод, який оцінює важливість ознаки шляхом випадкового перемішування її значень у тестовій вибірці. Якщо якість моделі (наприклад, F1-score) після перемішування сильно падає, ознака вважається важливою.

Для пояснення індивідуальних прогнозів (локальної інтерпретації) використовується метод SHAP (SHapley Additive exPlanations), який базується на теорії кооперативних ігор. SHAP розраховує внесок кожної конкретної ознаки у відхилення прогнозу для конкретного об'єкта від середнього прогнозу по датасету, дозволяючи детально розібрати причини розбіжностей між різними моделями

ХІД РОБОТИ

1. Первинний аналіз та очищення даних

Розпочав роботу з ініціалізації середовища, зафіксувавши індивідуальний SEED = 45. Далі завантажив датасет *Annealing* з OpenML та привів усі категоріальні ознаки до типу object. Датасет містить 898 рядків та 38 ознак.

Під час аналізу пропусків я виявив колонки, які були повністю порожніми або складалися лише з одного значення. Їх було видалено, оскільки вони не несуть корисної інформації для моделей

Щоб зрозуміти природу пропусків, я використав критерій Xi^2 для перевірки залежності між наявністю пропуску та цільовим класом

```
for col in top_3_missing:
    # Бінарний індикатор: 1 - пропуск, 0 - не пропуск
    missing_indicator = X[col].isna().astype(int)

    # Таблиця спряженості "пропуск/не пропуск" x "клас"
    contingency_table = pd.crosstab(missing_indicator, y)

    # Критерій Хі-квадрат
    chi2, p, dof, expected = chi2_contingency(contingency_table)
```

Лістинг 1. Перевірка механізму пропусків (Chi-square test)

Оскільки p-value виявилося значно меншим за 0.05, я відхилив гіпотезу про випадковість. Це означає, що механізм пропусків є MAR або MNAR (тобто сам факт пропуску залежить від класу і не є випадковим)

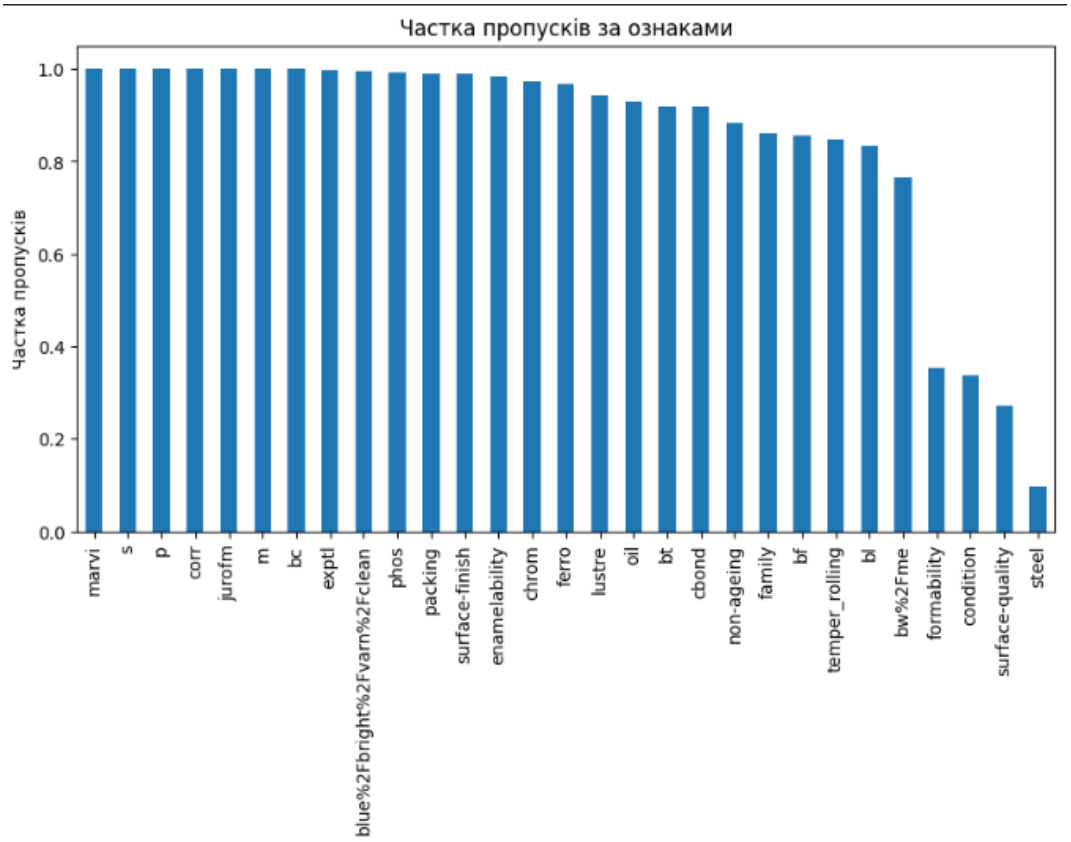


Рис. 1. Графік частки пропусків

```
•••  Аналіз механізму пропусків (Chi-square test):

     Ознака 'blue%2Fbright%2Fvarn%2Fclean':
     class                          1   2    3   5   U
     blue%2Fbright%2Fvarn%2Fclean
     0                              1   4    1   0   0
     1                              7  95  683  67  40
     p-value: 1.5146e-07
     Висновок: Пропуски пов'язані з класом. Це скоріше MAR/MNAR, а не MCAR.

     Ознака 'packing':
     class    1   2    3   5   U
     packing
     0        0   0    9   0   0
     1        8  99  675  67  40
     p-value: 5.8421e-01
     Висновок: Немає підстав відхиляти незалежність. Пропуски можуть бути MCAR.

     Ознака 'enamelability':
     class          1   2    3   5   U
     enamelability
     0              0  10    6   0   0
     1              8  89  678  67  40
     p-value: 5.2039e-09
     Висновок: Пропуски пов'язані з класом. Це скоріше MAR/MNAR, а не MCAR.

     Класи до кодування: ['1' '2' '3' '5' 'U']
     Класи після кодування: [0 1 2 3 4]
```

Рис. 2. Консольний вивід результатів Chi-square test

2. Поділ даних та створення пайплайнів

Розділив дані на тренувальну і тестову вибірки (80/20) зі стратифікацією

Щоб уникнути витоку даних (коли інформація з тесту потрапляє в тренування під час заповнення пропусків), я зібрав усю обробку в Pipeline Створив два варіанти:

- Варіант А: просто заповнення медіаною та модою.
- Варіант В: заповнення + створення додаткових колонок-індикаторів, які показують, чи був у цьому місці пропуск.

```python
def build_preprocessor(with_indicators: bool) -> ColumnTransformer:
    transformers = [
        ('num', SimpleImputer(strategy='median'),
         make_column_selector(dtype_include='number')),
        ('cat', Pipeline([
            ('impute', SimpleImputer(strategy='most_frequent')),
            ('onehot', OneHotEncoder(handle_unknown='ignore', sparse_output=False))
        ]), make_column_selector(dtype_exclude='number')),
    ]

    if with_indicators:
        transformers.append(
            ('miss', MissingIndicator(features='missing-only'),
             make_column_selector())
        )
    return ColumnTransformer(transformers)
```

Лістинг 2. Функція побудови препроцесора з індикаторами

3. Оцінка базових моделей

Навчив три моделі (Decision Tree, Random Forest, XGBoost) з базовими налаштуваннями і перевірив їх через 5-блочну крос-валідацію.

```
Оцінка моделей...

Таблиця: Таблиця 1: Крос-валідація, Варіант А (базова імпутація)
```

	Model	Accuracy	Macro F1	Balanced Acc
0	DT	0.9364 ± 0.0172	0.8131 ± 0.0928	0.8310 ± 0.1014
1	RF	0.9355 ± 0.0159	0.8581 ± 0.0884	0.8551 ± 0.0953
2	XGB	0.9410 ± 0.0201	0.8416 ± 0.1047	0.8370 ± 0.1022

```
Таблиця: Таблиця 2: Крос-валідація, Варіант В (з індикаторами пропусків)
```

	Model	Accuracy	Macro F1	Balanced Acc
0	DT	0.9865 ± 0.0082	0.9200 ± 0.0854	0.9178 ± 0.0908
1	RF	0.9875 ± 0.0102	0.9140 ± 0.0824	0.9107 ± 0.0890
2	XGB	0.9824 ± 0.0084	0.8999 ± 0.0850	0.8919 ± 0.0906

Рис.3. Таблиця 1 та Таблиця 2 (результати крос-валідації Варіант А і Варіант В)

Результати показали, що після додавання індикаторів пропусків (Варіант B) метрика Macro F1 помітно зросла. Це практично підтвердило мій здогад із першого пункту: пропуски несуть важливу інформацію

4. Підбір гіперпараметрів та вкладена крос-валідація

Для найкращого Варіанту B я запустив підбір параметрів через Optuna. Щоб моя оцінка якості не була оптимістично завищеною, я використав вкладену крос-валідацію (Nested CV)

У ній параметри підбираються лише на внутрішніх блоках, а тестуються на зовнішніх, які модель ще не бачила

```
for tr_idx, va_idx in outer.split(X_train, y_train):
    X_tr, y_tr = X_train.iloc[tr_idx], y_train[tr_idx]
    X_va, y_va = X_train.iloc[va_idx], y_train[va_idx]

    study = tune(name, X_tr, y_tr, n_trials=30)

    # Навчаємо модель з найкращими параметрами на X_tr
    model = build_pipeline(name, True, study.best_params).fit(X_tr, y_tr)

    # Оцінюємо на відкладеній частині зовнішнього блоку (X_va)
    score = f1_score(y_va, model.predict(X_va), average='macro')
    nested_scores.append(score)
```

Лістинг 3. Вкладена крос-валідація для неупередженої оцінки

```
Вкладена крос-валідація для DT...
Фінальна оптимізація на всій X_train для DT...
Вкладена крос-валідація для RF...
Фінальна оптимізація на всій X_train для RF...
Вкладена крос-валідація для XGB...
Фінальна оптимізація на всій X_train для XGB...

Таблиця 3: Вкладена крос-валідація та best_value (Варіант B)
```

	Model	Nested CV Macro F1	Study best_value
0	DT	0.9155 ± 0.0751	0.9404
1	RF	0.9172 ± 0.0767	0.9378
2	XGB	0.9256 ± 0.0781	0.9202

```
Таблиця 4: Найкращі гіперпараметри
DT: {'max_depth': 17, 'min_samples_split': 2, 'min_samples_leaf': 4, 'class_weight': None}
RF: {'n_estimators': 200, 'max_depth': 6, 'min_samples_leaf': 1, 'max_features': 'sqrt', 'class_weight': 'balanced_subsample'}
XGB: {'n_estimators': 339, 'max_depth': 6, 'learning_rate': 0.013773826655860183, 'subsample': 0.9548812950439447, 'colsample_bytree': 0.8967016511842438, 'min_child_weight': 1.3241350313924893}
```

Рис.4. Таблиця 3 (Вкладена крос-валідація та best_value)

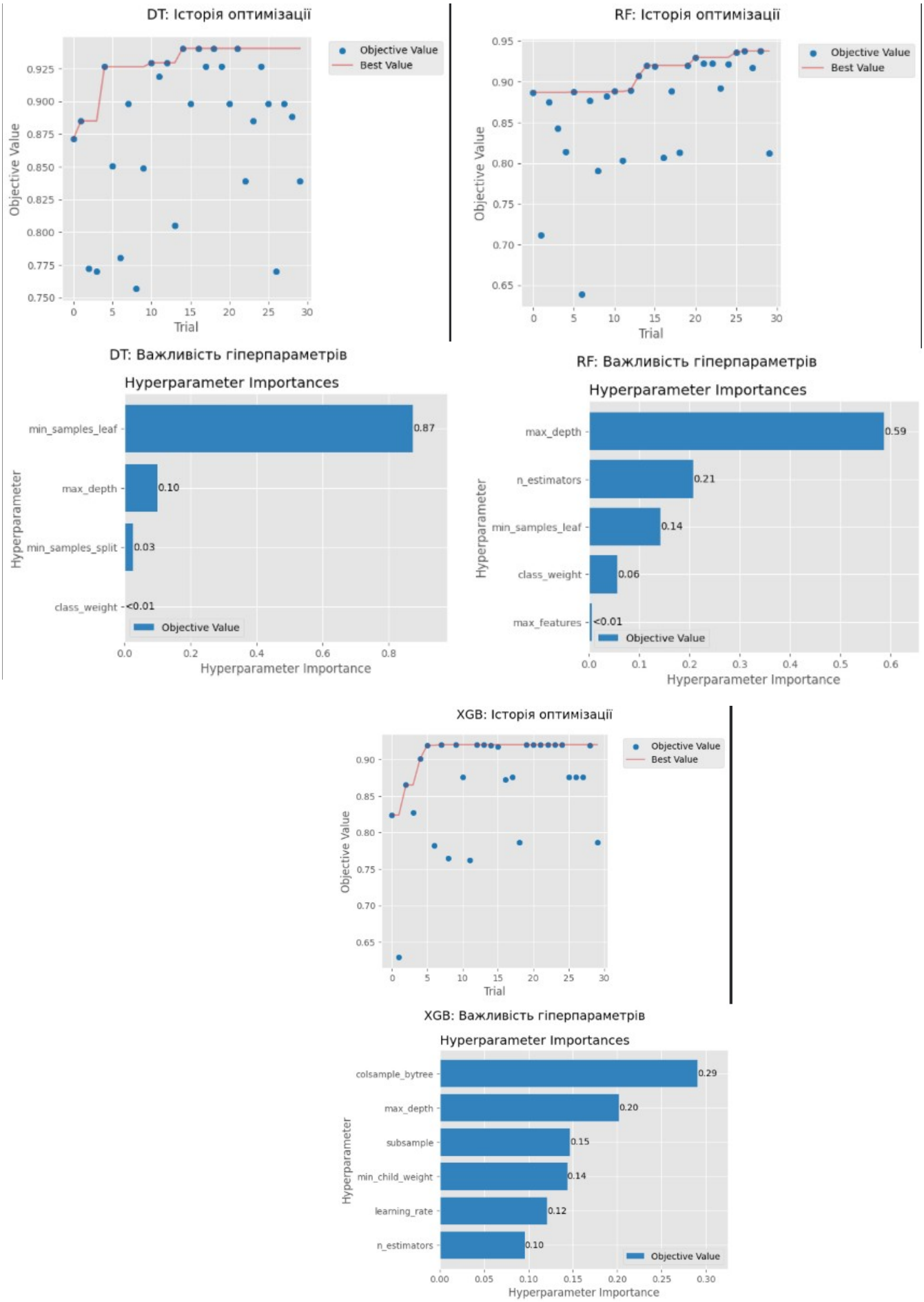


Рис.5. Графіки Optuna "Історія оптимізації" та "Важливість гіперпараметрів" для кожної моделі

Як видно з Таблиці 3, значення best_value виявилося вищим за чесну оцінку Nested CV. Це доводить, що якби я просто взяв best_value, моя оцінка була б оптимістично зміщеною

5. Статистичне порівняння

Порівняв результати моделей за допомогою скоригованого t-тесту Наде-Бенжіо, який правильно працює з крос-валідацією, враховуючи перетин навчальних даних (дисперсію)

```
... Збір метрик для статистичного порівняння...

Таблиця 5: Результати статистичного порівняння
```

	Порівняння	Різниця (d_mean)	t-статистика	p-value	Висновок
0	DT: Вар B - Вар A	0.1068	2.0860	5.5753e-02	Незначуща
1	RF: Вар B - Вар A	0.0558	4.0684	1.1511e-03	Значуща
2	XGB: Вар B - Вар A	0.0584	2.2670	3.9756e-02	Значуща
3	Топ-2 (Вар B): DT - RF	0.0060	0.3379	7.4046e-01	Незначуща

Рис.6. Таблиця 5 (Результати статистичного порівняння)

Тест підтвердив, що приріст від індикаторів пропусків є статистично значущим (p-value < 0.05) для ансамблевих моделей. А от різниця між найкращими оптимізованими моделями виявилася незначущою

6. Фінальна оцінка на тестовій вибірці

Застосував готові моделі до тестової вибірки. Найкращою виявилася XGBoost

Таблиця 6: Якість фінальних моделей на тестовій вибірці

	Model	Accuracy	Macro F1	Balanced Acc
0	DT	0.9833	0.9149	0.8971
1	RF	0.9833	0.9838	0.9956
2	XGB	0.9944	0.9859	0.9750

Найкраща модель: XGB

Classification Report:

	precision	recall	f1-score	support
1	1.00	1.00	1.00	2
2	1.00	1.00	1.00	20
3	0.99	1.00	1.00	137
5	1.00	1.00	1.00	13
U	1.00	0.88	0.93	8
accuracy			0.99	180
macro avg	1.00	0.97	0.99	180
weighted avg	0.99	0.99	0.99	180

Рис.7. Таблиця 6 (Оцінка на тесті) та Classification Report

З матриці плутанини видно, що XGBoost зробив лише одну помилку: відніс один зразок найменшого класу U до найбільшого класу 3. Загальна якість дуже висока

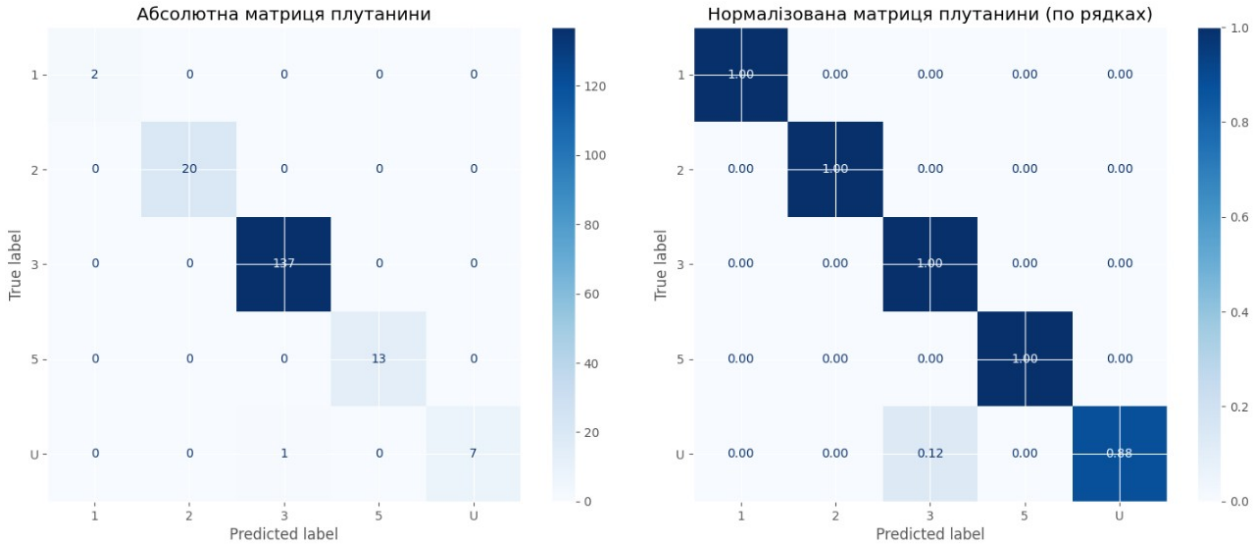


Рис.8. Матриці плутанини (абсолютна та нормалізована)

7. Важливість ознак та аналіз розбіжностей (SHAP)

Побудував графіки MDI (важливість усередині дерев на оброблених ознаках) та Permutation Importance (важливість на початкових ознаках)

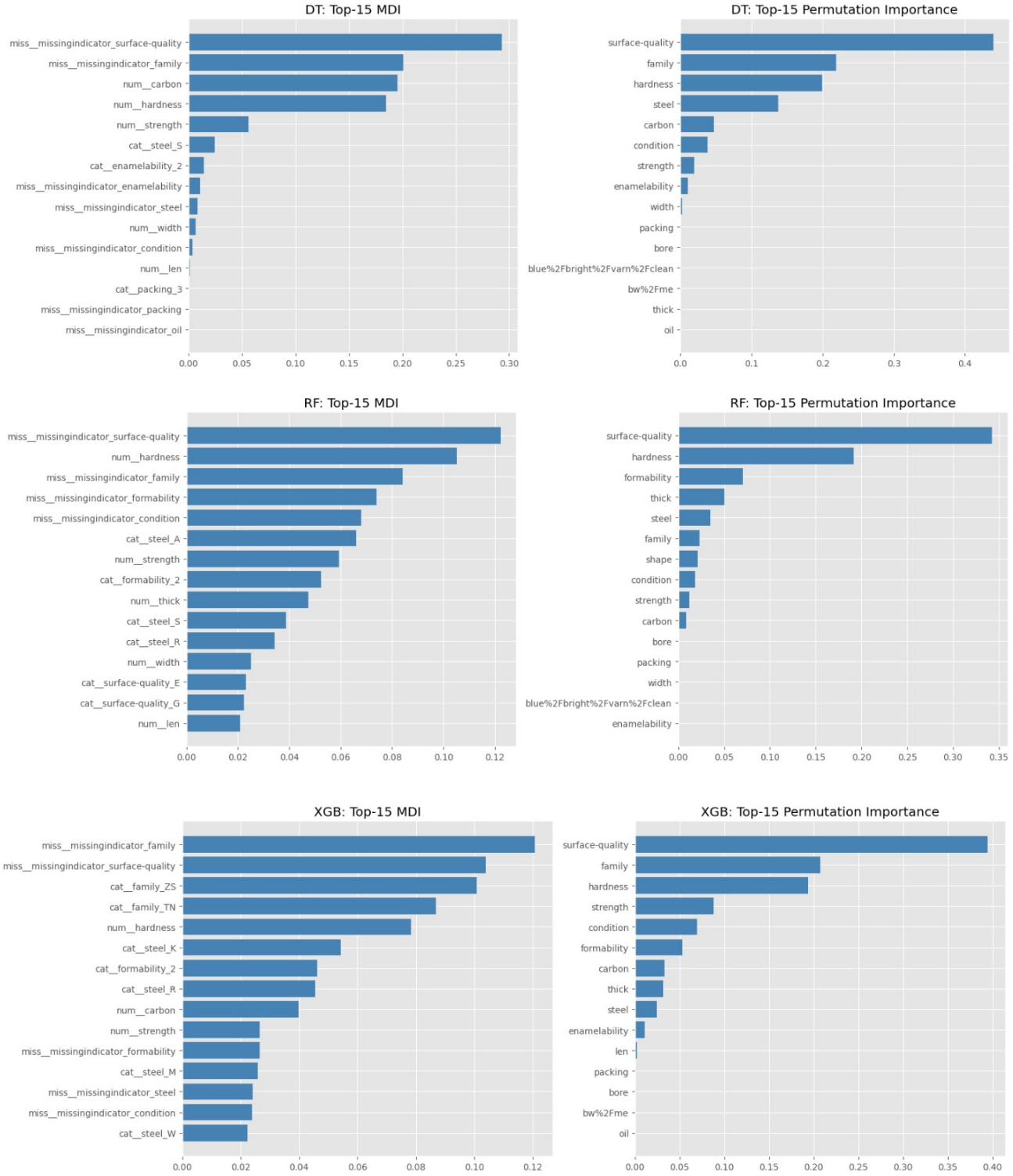


Рис.9. Графіки Top-15 MDI та Permutation Importance

На графіках MDI індикатори пропусків (miss_missingindicator_*) стабільно входять у Топ-15, що доводить їхню важливість. На графіках Permutation Importance лідирують фізичні характеристики (surface-quality, family, hardness)

Оскільки моделі працювали дуже добре, я знайшов об'єкти розбіжностей за допомогою прогнозів на крос-валідації і побудував пояснення через SHAP

```
explainer = shap.TreeExplainer(clf)
shap_values = explainer.shap_values(X_target_processed)

predicted_class_enc = y_pred_oof['XGB'][target_idx]
predicted_class_name = le.inverse_transform([predicted_class_enc])[0]
print(f"\nПрогноз XGBoost: клас {predicted_class_name}")

# ВИПРАВЛЕННЯ: універсальна обробка розмірності SHAP значень
if isinstance(shap_values, list):
    shap_vals_class = shap_values[predicted_class_enc][0]
elif len(shap_values.shape) == 3:
    # Формат (n_samples, n_features, n_classes)
    shap_vals_class = shap_values[0, :, predicted_class_enc]
else:
    # Формат для бінарної/деяких інших версій (n_samples, n_features)
    shap_vals_class = shap_values[0]

# Топ-5 ознак
top_indices = np.argsort(np.abs(shap_vals_class))[-5:][::-1]
```

Лістинг 4. Розрахунок SHAP-значень

SHAP чітко показав, що моделі по-різному реагували на заповнені пропуски поблизу меж класів: дерево приймало рішення різко через жорсткі пороги, а ансамблі давали згладжений результат

Знайдено розбіжностей: 10

Таблиця 7: Об'єкти розбіжностей

	Об'єкт	Справжній клас	DT	RF	XGB
0	6	3	2	3	3
1	25	1	1	1	3
2	54	3	2	3	3

```
Детальний аналіз для об'єкта #6

Ознаки об'єкта (до імпутації):
family: Пропуск (буде імпутовано)
steel: V
carbon: 0
hardness: 0
condition: S
formability: 2
strength: 0
surface-quality: Пропуск (буде імпутовано)
enamelability: 1
bw%2Fme: Пропуск (буде імпутовано)
blue%2Fbright%2Fvarn%2Fclean: Пропуск (буде імпутовано)
shape: COIL
thick: 0.699
width: 609.9
len: 0
oil: Пропуск (буде імпутовано)
bore: 0
packing: Пропуск (буде імпутовано)

Прогноз XGBoost: клас 3

Топ-5 ознак з найбільшим внеском (SHAP):
Ознака: miss__missingindicator_surface-quality | Значення: 1.0000 | SHAP внесок: -1.4397
Ознака: cat__steel_V | Значення: 1.0000 | SHAP внесок: 1.3621
Ознака: num__carbon | Значення: 0.0000 | SHAP внесок: -0.2973
Ознака: cat__enamelability_1 | Значення: 1.0000 | SHAP внесок: 0.2788
Ознака: num__thick | Значення: 0.6990 | SHAP внесок: 0.2099
```

Рис.10. Таблиця 7 (Об'єкти розбіжностей) та консольний вивід Топ-5 ознак SHAP

ВІДПОВІДІ НА ПИТАННЯ

1. **RF vs Decision Tree:** Звичайне дерево рішень схильне до перенавчання (має високу дисперсію). Випадковий ліс будує багато дерев на різних підвибірках даних і ознак, а потім усереднює їхні прогнози. Це суттєво зменшує дисперсію. Декореляція (щоб дерева були різними) потрібна для того, щоб їхні індивідуальні помилки успішно компенсувалися при усередненні.
2. **Бустинг vs Беггінг:** Беггінг (Random Forest) будує дерева незалежно і паралельно, зменшуючи розкид відповідей. Бустинг (XGBoost) будує дерева послідовно: кожне нове дерево намагається виправити помилки попередніх, зменшуючи зміщення. Через це бустинг швидше перенавчається, якщо додати забагато дерев, бо починає "запам'ятовувати" шум.

3. **Метрики в умовах дисбалансу:** Звичайна Accuracy може показувати 95%, якщо мажоритарний клас становить 95% даних, навіть якщо модель ігнорує всі інші класи. Macro F1 рахує точність для кожного класу окремо і знаходить середнє, змушуючи модель однаково добре працювати і з великими, і з малими класами
4. **Механізми пропусків:** MCAR – пропуск виник цілком випадково (наприклад, збій датчика). MAR – ймовірність пропуску залежить від інших відомих колонок. MNAR – пропуск залежить від самого невідомого значення. Індикатори пропусків найкорисніші саме при MAR та MNAR
5. **Витік даних під час імпутації:** Якщо заповнити пропуски середнім значенням по всьому датасету ДО того, як ми розіб'ємо його на train і test, тестові дані "зіллються" в навчальні. Щоб цього уникнути, імпутацію потрібно робити суворо всередині Pipeline, щоб середнє рахувалося лише на тренувальному блоці
6. **Оптимістичне зміщення:** Значення best_value з Optuna – це максимум, обраний серед багатьох варіантів, які підганялися під валідаційну вибірку, тому він завищений. Вкладена крос-валідація відокремлює процес підбору параметрів від процесу фінальної оцінки, тестуючи модель на повністю незалежних даних
7. **Тест Наде-Бенжіо:** Звичайний парний t-тест вважає всі оцінки незалежними. Але в крос-валідації навчальні блоки містять одні й ті самі дані, тому оцінки залежні (дисперсія занижується). Тест Наде-Бенжіо вносить поправку в формулу, враховуючи відношення тестової та тренувальної вибірок, роблячи порівняння математично коректним
8. **MDI vs Permutation Importance:** MDI рахується всередині дерева під час навчання і може помилково надавати високу важливість ознакам, які мають багато унікальних значень. Permutation Importance рахується на тестових даних шляхом випадкового перемішування значень колонки: якщо точність після цього сильно падає, отже, ознака була важливою. Це більш чесний метод
9. **SHAP-значення:** Це спосіб пояснити рішення моделі для конкретного об'єкта. Властивість адитивності означає, що якщо додати SHAP-внески всіх ознак об'єкта до базового середнього прогнозу моделі, ми отримаємо фінальний прогноз моделі для цього ж об'єкта

10. **XGBoost і пропуски:** XGBoost вміє працювати з NaN без попереднього заповнення. Під час побудови гілок він пробує відправити всі пропущені значення спочатку вліво, потім вправо, і автоматично обирає той напрямок, який дає кращий результат (меншу похибку).
11. **Переваги Optuna:** Optuna використовує "розумний" байєсівський пошук (TPE), який аналізує попередні спроби і вибирає наступні параметри там, де очікується кращий результат, що набагато швидше за випадковий пошук. Також вона вміє достроково зупиняти (прунінг) ті спроби навчання, які від самого початку показують погані результати

Висновок

У процесі виконання лабораторної роботи я порівняв алгоритми Decision Tree, Random Forest та XGBoost на даних із пропусками. Найкращою моделлю за метрикою Macro F1 на тестовій вибірці виявився XGBoost, хоча статистичний тест Наде-Бенжіо показав, що його перевага над Random Forest не є статистично значущою

Додавання індикаторів пропусків (Варіант B) помітно покращило якість роботи моделей. Це повністю узгоджується з моїм початковим аналізом механізму пропусків (MAR/MNAR) та графіками MDI, де ці індикатори стабільно потрапляли в Топ-15 найважливіших ознак

Порівняння метрик показало, що значення best_value з Optuna завжди було вищим за результати вкладеної крос-валідації, що практично доводить ризик оптимістичного зміщення при звичайному підборі параметрів

Найбільший приріст після оптимізації отримав XGBoost, оскільки цей алгоритм має багато гіперпараметрів і дуже чутливий до їх правильного налаштування. Найскладнішим для класифікації виявився найменший клас U, об'єкти якого іноді помилково відносилися до мажоритарного класу 3 саме через сильний дисбаланс прикладів у датасеті

Аналіз розбіжностей за допомогою SHAP-значень наочно показав, що звичайне дерево рішень різко реагує на штучно імпутовані значення через жорсткі пороги розбиття, тоді як ансамблі усереднюють ці відхилення і приймають більш згладжені рішення

Враховуючи високу якість, стабільність та наявність вбудованої обробки пропусків, для практичного використання я б обрав XGBoost, оскільки він показав найкращу загальну точність при цілком прийнятних обчислювальних витратах